

El examen tiene una duración de 1:30 horas.

Ejercicio de Programación (6p)

Una persona desea vender toda su colección de Funko Pops de películas de Marvel; todos los funkos que quiere vender son diferentes. Por este motivo, una vez que se vende un Funko Pop, no se puede volver a vender. Se pide implementar una aplicación SPA para vender Funko Pops de películas de Marvel. La aplicación debe ser capaz de listar, vender y enviar los Funko Pops a los clientes. Las funcionalidades específicas de la aplicación se definen a continuación:

- Funcionalidades:
 - La aplicación dispone de dos tipos de usuarios: cliente y administrador. El usuario cliente será capaz de comprar los Funko Pops, mientras que el usuario administrador se encargará de gestionar el envío de los productos. Para utilizar la aplicación, es necesario iniciar sesión como cliente o administrador; no se considera ninguna funcionalidad para usuarios no registrados.
 - Los dos usuarios entran por defecto a la página principal de la aplicación. En esta página se debe mostrar el título “Marvel Pop Paradise” y, a continuación, una lista con cada uno de los productos almacenados en la base de datos. Para cada producto, se mostrarán el nombre, el precio y un botón “Comprar” que permitirá su compra. Además, si el usuario es administrador, se le debe mostrar un enlace para ir a la página del administrador.
 - Los productos de la lista se mostrarán ordenados por su precio, de mayor a menor. Además, se aplicará un formato visual específico según el precio del producto: si el precio es igual o mayor a 100 €, se pintará de color rojo; si es menor de 100 €, se pintará de color verde.
 - Cuando un usuario pulse el botón “Comprar”, se abrirá una nueva página para ingresar la dirección de envío. En esta página, se visualizará el título “Realizar compra”, seguido de un formulario para indicar la dirección de envío y un botón “Confirmar”. Al pulsar “Confirmar”, el producto se considerará comprado y se redirigirá al usuario a la página principal, en la cual ya no aparecerá el producto que acaba de comprar.
 - La página del administrador mostrará el título “Productos a enviar”, seguido de una lista con cada uno de los productos ya comprados. Para cada producto, se visualizarán el nombre, la dirección de envío y un botón “Enviado”. Al pulsar el botón “Enviado”, se asumirá que el administrador ha enviado el producto, considerándose, por tanto, enviado, y ese producto ya no aparecerá en la lista de productos a enviar.
 - Es importante que un cliente no pueda realizar las operaciones pertenecientes a un administrador. Para ello, se deben incluir los mecanismos adecuados en el código.
 - A continuación se proporciona el esqueleto del archivo “WebSecurityConfig.java” del backend, por si fuera necesario su uso:

```
public SecurityFilterChain apiFilterChain(HttpSecurity http) {  
    http.authorizeHttpRequests(authorize -> authorize  
        // PRIVATE ENDPOINTS  
        .requestMatchers(...).hasRole(...)  
    );  
}
```

- Los roles para el cliente y el administrador son “ROLE_CLIENT” y “ROLE_ADMIN”, respectivamente.
- Es necesario definir las rutas en el frontend SPA. Para ello, basta con que el archivo “app.routing.ts” contenga un array como el siguiente, con las rutas y los componentes correctamente definidos:

```
const appRoutes: Routes = [  
    { path: "...", component: ... },  
]
```

- Cuestiones de implementación:
 - Se deberá implementar una API REST y un frontend de tipo SPA. No se debe implementar una interfaz web con arquitectura MVC tradicional.
 - Se asume que la base de datos a la que se conecta la aplicación ya dispone de un usuario cliente y un usuario administrador.
 - Se considerará que cualquier entidad, controlador, método o servicio que gestione usuarios está disponible para su uso en el backend, por lo tanto, no es necesario implementarlo.
 - Se asume que la página de login ya está disponible y no es necesario implementarla.
 - Se asume que la base de datos a la que se conecta la aplicación ya contiene la información con los productos. La aplicación no tiene la capacidad de dar de alta nuevos productos.
 - No es necesario usar ningún tipo de librería de componentes.
 - El código deberá estar en inglés. Lo único que puede estar en castellano son los textos que aparecen en la interfaz de usuario.

Se pide:

- **A) Implementar el backend con Spring MVC, SpringData y JPA. (2p)**
 - No es necesario escribir el fichero “pom.xml”, se puede asumir que tiene todas las dependencias correspondientes.
 - No es necesario escribir el fichero “application.properties”, se puede asumir que la aplicación está conectada a una base de datos externa correctamente configurada.
 - No es necesario escribir la clase Application.
 - Es necesario escribir TODOS los demás ficheros de la aplicación.
 - No es necesario incluir los imports en los ficheros Java.
 - No es necesario implementar los getter y setter de las clases Java. Se pueden dejar indicados con un comentario.
 - La API REST debe respetar las principios de diseño: formato de las URLs, uso de los códigos de estado, cabeceras, métodos, etc.
- **C) Implementar el frontend usando Angular (2p)**
 - Se puede asumir que se dispone de un proyecto Angular creado con “ng new” con todos los ficheros necesarios (index.html, angular.cli, package.json, tsconfig.json, app.module.ts, etc.).
 - Hay que escribir el resto de ficheros necesarios para implementar los componentes, servicios y configuración de rutas.
 - No es necesario incluir los imports en los ficheros TypeScript.
 - Se usará HTML plano en el template de los componentes (sin ninguna librería de componentes como ng-bootstrap o angular-material).
 - Se asumirá que el proxy está correctamente configurado y se pueden usar URLs relativas para acceder a la API REST del backend.
- **D) Empaquetar la aplicación en un contenedor Docker (2p)**
 - Se desea crear una imagen Docker específica para servir la aplicación SPA (frontend) a través de HTTP. Es decir, la aplicación SPA no será servida por HTTP desde el backend Spring Boot en la carpeta “static”. En su lugar, la aplicación se publicará en la raíz del dominio, por ejemplo, en <http://www.miweb.com/>.
 - Para servir la aplicación SPA por HTTP, se utilizará un servidor NGINX. Para crear una imagen Docker basada en NGINX que sirva contenido estático a través de HTTP, bastará con crear una imagen basada en nginx:1.26.0 y copiar el contenido a servir en la carpeta “/usr/share/nginx/html”. No será necesario indicar el comando CMD, ya que está definido en la imagen base de NGINX.
 - Se pide crear un Dockerfile Multi-stage que primero construya la aplicación para su publicación y luego genere la imagen final basada en NGINX. Para ello, se debe utilizar las imágenes disponibles en DockerHub, node:20.12.2 y nginx:1.26.0.
 - También se pide crear un script (build.sh o build.ps1 o build.bat) para la construcción y publicación de la imagen Docker utilizando el Dockerfile proporcionado. El script debe contener los comandos necesarios para construir la imagen Docker con el nombre “daw/marvel-pop-paradise:1.0.0” y para

su publicación en DockerHub. Se asume que el sistema está logueado en DockerHub con los permisos necesarios para publicar la imagen.

- Se asumirá que el código de la aplicación SPA está disponible en una carpeta llamada “frontend”, y que tanto el Dockerfile como el script están en la carpeta raíz del proyecto.

Instrucciones de entrega

El esquema para las carpetas del examen debe ser el siguiente:

- El código del backend debe estar en una carpeta llamada “backend”.
- El código del frontend debe estar en una carpeta llamada “frontend”.
- El código de Docker debe estar en una carpeta llamada “docker”.

Solo se debe subir un fichero como entrega del examen. Dicho fichero tiene que ser un ZIP creado con “7-Zip”. El nombre del ZIP debe ser “PPG-Examen-Mayo.zip”, donde “PPG” son las iniciales del alumno “Pepito Pérez González”.